

TUẦN 4: Database Integration - Tài Liệu Lý Thuyết Chi Tiết

Giáo viên hướng dẫn: Senior Java Developer Cấp độ: Nâng cao (Tiếp theo Tuần 1-3) Thời lượng: 1 tuần (7 phiên)

Nội dung Tuần 4

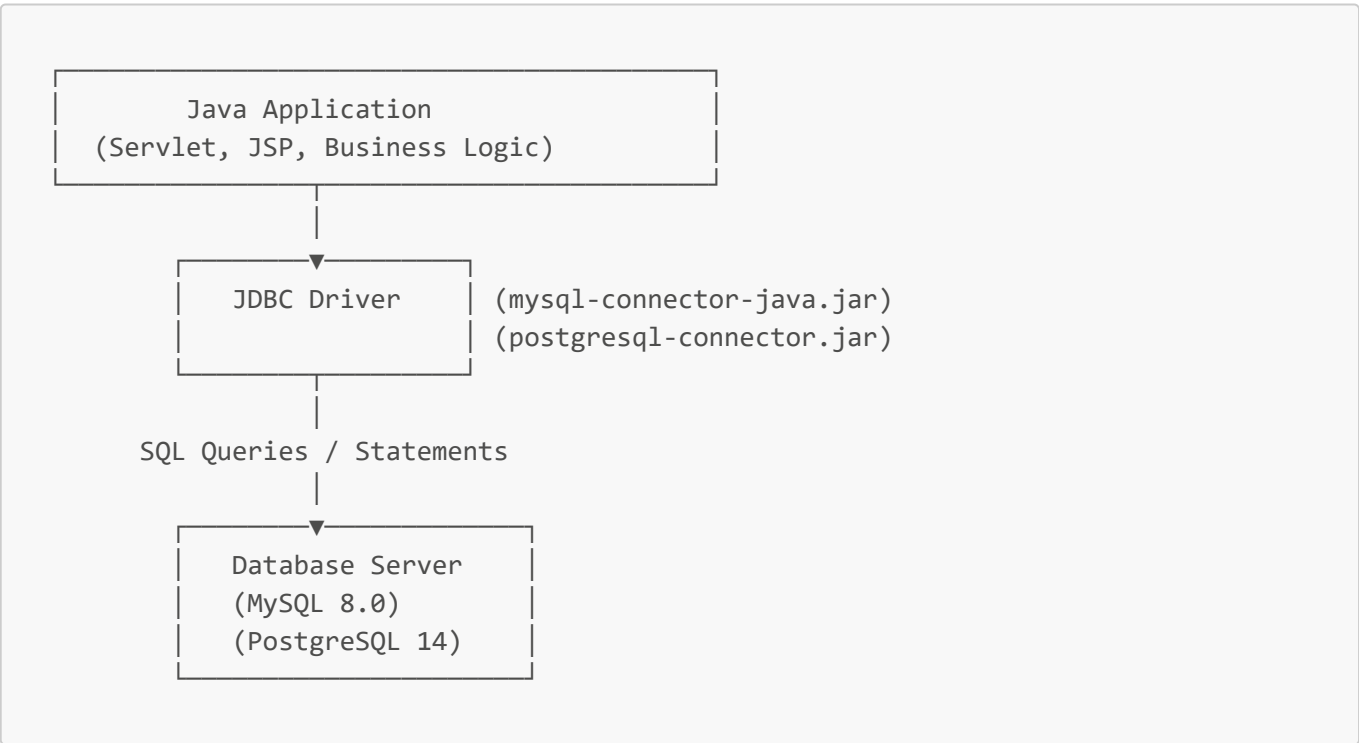
- **Session 22:** JDBC Basics (Kết nối Database)
 - **Session 23:** DAO Pattern
 - **Session 24:** MySQL/PostgreSQL Setup
 - **Session 25:** Integrate JDBC with Servlet/JSP
 - **Session 26:** PreparedStatement & SQL Injection
 - **Session 27:** Connection Pool
 - **Session 28:** Mini Project
-

SESSION 22: JDBC Basics

22.1 JDBC là gì?

JDBC = **J**ava **D**atabase **C**onnectivity

Là một API giúp Java kết nối và tương tác với database.



22.2 JDBC APIs Chính

JDBC APIs (java.sql)

1. DriverManager
 - └─ getConnection()
2. Connection
 - └─ createStatement()
 - └─ prepareStatement()
 - └─ commit()
 - └─ close()
3. Statement / PreparedStatement
 - └─ executeQuery() (SELECT)
 - └─ executeUpdate() (INSERT/UPDATE)
 - └─ executeBatch() (nhiều query)
4. ResultSet
 - └─ next()
 - └─ getString(), getInt()
 - └─ wasNull()
5. Exception Handling
 - └─ SQLException

22.3 Các Bước Sử Dụng JDBC

STEP 1: Load JDBC Driver

```
Class.forName(  
    "com.mysql.cj.jdbc.Driver"  
);
```

↓

STEP 2: Tạo Connection

```
Connection conn =  
    DriverManager.getConnection  
    (url, user, pass);
```

↓

STEP 3: Tạo Statement

```
Statement stmt =  
    conn.createStatement();
```

**STEP 4: Thực thi Query**

```
ResultSet rs =  
    stmt.executeQuery(sql);
```

**STEP 5: Xử lý Kết quả**

```
while (rs.next()) {  
    String name = rs.  
        getString("name");  
}
```

**STEP 6: Đóng Tài nguyên**

```
rs.close();  
stmt.close();  
conn.close();
```

22.4 Connection String (URL)

MySQL

```
String url = "jdbc:mysql://localhost:3306/database_name";  
// jdbc:mysql://host:port/database  
  
String url = "jdbc:mysql://localhost:3306/database_name?  
useSSL=false&serverTimezone=UTC&characterEncoding=UTF-8";  
// Với options
```

PostgreSQL

```
String url = "jdbc:postgresql://localhost:5432/database_name";  
// jdbc:postgresql://host:port/database
```

SQL Server

```
String url = "jdbc:sqlserver://localhost:1433;databaseName=database_name";
```

22.5 Ví dụ Cơ Bản JDBC

Kết nối Database

```
import java.sql.*;

public class JDBCExample {

    public static void main(String[] args) {
        String url = "jdbc:mysql://localhost:3306/mydb";
        String user = "root";
        String password = "password";

        Connection conn = null;

        try {
            // STEP 1: Load Driver (thường không cần trong Java 6+)
            Class.forName("com.mysql.cj.jdbc.Driver");

            // STEP 2: Tạo Connection
            conn = DriverManager.getConnection(url, user, password);
            System.out.println("☑ Connected successfully!");

        } catch (ClassNotFoundException e) {
            System.out.println("✗ JDBC Driver not found");
            e.printStackTrace();

        } catch (SQLException e) {
            System.out.println("✗ Connection failed");
            e.printStackTrace();

        } finally {
            // STEP 6: Đóng connection
            if (conn != null) {
                try {
                    conn.close();
                } catch (SQLException e) {
                    e.printStackTrace();
                }
            }
        }
    }
}
```

22.6 SELECT Query

Ví dụ: Lấy tất cả users

```
String url = "jdbc:mysql://localhost:3306/mydb";
String user = "root";
String password = "password";
```

```
Connection conn = DriverManager.getConnection(url, user, password);
Statement stmt = conn.createStatement();

// Thực thi SELECT query
ResultSet rs = stmt.executeQuery("SELECT id, name, email FROM users");

// Duyệt kết quả
while (rs.next()) {
    int id = rs.getInt("id");           // Lấy int
    String name = rs.getString("name"); // Lấy String
    String email = rs.getString("email");

    System.out.println("ID: " + id + ", Name: " + name + ", Email: " + email);
}

// Đóng tài nguyên
rs.close();
stmt.close();
conn.close();
```

Ví dụ: Lấy user theo ID

```
String sql = "SELECT id, name, email FROM users WHERE id = 5";

ResultSet rs = stmt.executeQuery(sql);

if (rs.next()) {
    String name = rs.getString("name");
    String email = rs.getString("email");
    System.out.println("Name: " + name + ", Email: " + email);
} else {
    System.out.println("User not found");
}
```

22.7 INSERT/UPDATE/DELETE Query

INSERT

```
String sql = "INSERT INTO users (name, email, phone) VALUES ('Nhân',
'nhan@example.com', '0912345678')";

Statement stmt = conn.createStatement();
int rowsAffected = stmt.executeUpdate(sql);

System.out.println(rowsAffected + " rows inserted");
```

UPDATE

```
String sql = "UPDATE users SET email = 'new@example.com' WHERE id = 5";

int rowsAffected = stmt.executeUpdate(sql);
System.out.println(rowsAffected + " rows updated");
```

DELETE

```
String sql = "DELETE FROM users WHERE id = 5";

int rowsAffected = stmt.executeUpdate(sql);
System.out.println(rowsAffected + " rows deleted");
```

22.8 ResultSet Methods

Method	Mô tả
next()	Di chuyển con trỏ đến hàng tiếp theo
getString(columnName)	Lấy giá trị chuỗi
getInt(columnName)	Lấy giá trị số nguyên
getDouble(columnName)	Lấy giá trị số thực
getDate(columnName)	Lấy giá trị ngày
getBoolean(columnName)	Lấy giá trị Boolean
wasNull()	Kiểm tra giá trị NULL
first()	Chuyển đến hàng đầu tiên
last()	Chuyển đến hàng cuối cùng
close()	Đóng ResultSet

22.9 Exception Handling

```
Connection conn = null;
Statement stmt = null;
ResultSet rs = null;

try {
    conn = DriverManager.getConnection(url, user, password);
    stmt = conn.createStatement();
    rs = stmt.executeQuery(sql);
}
```

```
        while (rs.next()) {
            // Xử lý
        }

    } catch (SQLException e) {
        System.err.println("SQL Error: " + e.getMessage());
        e.printStackTrace();
    } finally {
        // Đóng tất cả tài nguyên
        try {
            if (rs != null) rs.close();
            if (stmt != null) stmt.close();
            if (conn != null) conn.close();
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

Try-with-resources (Java 7+)

```
// Tự động đóng tài nguyên
try (Connection conn = DriverManager.getConnection(url, user, password);
     Statement stmt = conn.createStatement();
     ResultSet rs = stmt.executeQuery(sql)) {

    while (rs.next()) {
        // Xử lý
    }

} catch (SQLException e) {
    e.printStackTrace();
}
// Auto-closed at the end of try block
```

Tóm tắt Session 22

JDBC Basics:

- ☒ Load Driver → Create Connection → Create Statement → Execute Query → Process Result → Close
 - ☒ executeQuery() cho SELECT
 - ☒ executeUpdate() cho INSERT/UPDATE/DELETE
 - ☒ ResultSet để xử lý dữ liệu
 - ☒ Exception handling important
-
-

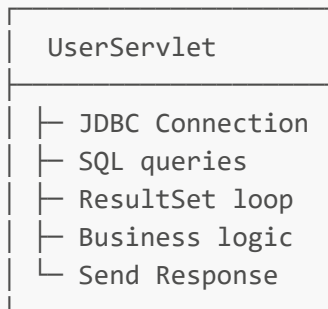
SESSION 23: DAO Pattern

23.1 DAO là gì?

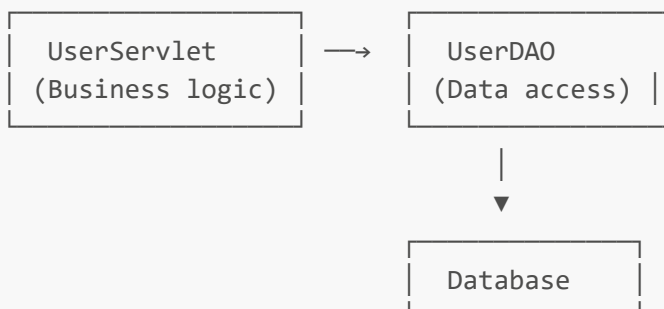
DAO = **D**ata **A**ccess **O**bject

Là một design pattern tách logic truy cập database khỏi business logic.

☒ Không DAO (Code lộn xộn):



☒ Với DAO (Code sạch):



23.2 DAO Pattern Cấu trúc

Interface - UserDao.java

```
package com.example.dao;

import com.example.model.User;
import java.util.List;

public interface UserDao {

    /**
     * Lấy tất cả users
     */
    List<User> findAll() throws Exception;
```



```
/**
 * Lấy user theo ID
 */
User findById(int id) throws Exception;

/**
 * Lấy user theo email
 */
User findByEmail(String email) throws Exception;

/**
 * Thêm user mới
 */
int insert(User user) throws Exception;

/**
 * Cập nhật user
 */
boolean update(User user) throws Exception;

/**
 * Xóa user
 */
boolean delete(int id) throws Exception;

/**
 * Đếm tất cả users
 */
int count() throws Exception;
}
```

Implementation - UserDaoImpl.java

```
package com.example.dao.impl;

import com.example.dao.UserDAO;
import com.example.model.User;
import java.sql.*;
import java.util.*;

public class UserDaoImpl implements UserDAO {

    private String url = "jdbc:mysql://localhost:3306/mydb";
    private String user = "root";
    private String password = "password";

    /**
     * Lấy Connection
     */
    private Connection getConnection() throws SQLException {
        return DriverManager.getConnection(url, user, password);
    }
}
```

```
}

@Override
public List<User> findAll() throws Exception {
    List<User> users = new ArrayList<>();
    String sql = "SELECT id, name, email, phone FROM users";

    try (Connection conn = getConnection();
        Statement stmt = conn.createStatement();
        ResultSet rs = stmt.executeQuery(sql)) {

        while (rs.next()) {
            User user = mapResultSetToUser(rs);
            users.add(user);
        }
    }

    return users;
}

@Override
public User findById(int id) throws Exception {
    String sql = "SELECT id, name, email, phone FROM users WHERE id = ?";

    try (Connection conn = getConnection();
        PreparedStatement pstmt = conn.prepareStatement(sql)) {

        pstmt.setInt(1, id);

        try (ResultSet rs = pstmt.executeQuery()) {
            if (rs.next()) {
                return mapResultSetToUser(rs);
            }
        }
    }

    return null;
}

@Override
public User findByEmail(String email) throws Exception {
    String sql = "SELECT id, name, email, phone FROM users WHERE email = ?";

    try (Connection conn = getConnection();
        PreparedStatement pstmt = conn.prepareStatement(sql)) {

        pstmt.setString(1, email);

        try (ResultSet rs = pstmt.executeQuery()) {
            if (rs.next()) {
                return mapResultSetToUser(rs);
            }
        }
    }
}
```

```
        return null;
    }

    @Override
    public int insert(User user) throws Exception {
        String sql = "INSERT INTO users (name, email, phone) VALUES (?, ?, ?)";

        try (Connection conn = getConnection();
            PreparedStatement pstmt = conn.prepareStatement(sql,
                Statement.RETURN_GENERATED_KEYS)) {

            pstmt.setString(1, user.getName());
            pstmt.setString(2, user.getEmail());
            pstmt.setString(3, user.getPhone());

            pstmt.executeUpdate();

            // Lấy auto-generated ID
            try (ResultSet generatedKeys = pstmt.getGeneratedKeys()) {
                if (generatedKeys.next()) {
                    return generatedKeys.getInt(1);
                }
            }
        }

        return -1;
    }

    @Override
    public boolean update(User user) throws Exception {
        String sql = "UPDATE users SET name = ?, email = ?, phone = ? WHERE id = ?";

        try (Connection conn = getConnection();
            PreparedStatement pstmt = conn.prepareStatement(sql)) {

            pstmt.setString(1, user.getName());
            pstmt.setString(2, user.getEmail());
            pstmt.setString(3, user.getPhone());
            pstmt.setInt(4, user.getId());

            int rowsAffected = pstmt.executeUpdate();
            return rowsAffected > 0;
        }
    }

    @Override
    public boolean delete(int id) throws Exception {
        String sql = "DELETE FROM users WHERE id = ?";

        try (Connection conn = getConnection();
            PreparedStatement pstmt = conn.prepareStatement(sql)) {
```

```

        pstmt.setInt(1, id);

        int rowsAffected = pstmt.executeUpdate();
        return rowsAffected > 0;
    }
}

@Override
public int count() throws Exception {
    String sql = "SELECT COUNT(*) FROM users";

    try (Connection conn = getConnection();
        Statement stmt = conn.createStatement();
        ResultSet rs = stmt.executeQuery(sql)) {

        if (rs.next()) {
            return rs.getInt(1);
        }

        return 0;
    }

    /**
     * Helper: Map ResultSet sang User object
     */
    private User mapResultSetToUser(ResultSet rs) throws SQLException {
        User user = new User();
        user.setId(rs.getInt("id"));
        user.setName(rs.getString("name"));
        user.setEmail(rs.getString("email"));
        user.setPhone(rs.getString("phone"));
        return user;
    }
}

```

23.3 Sử dụng DAO

Trong Servlet

```

@WebServlet("/user")
public class UserServlet extends HttpServlet {

    private UserDAO userDAO = new UserDAOImpl();

    protected void doGet(HttpServletRequest request,
                        HttpServletResponse response)
        throws ServletException, IOException {

        try {

```

```
String action = request.getParameter("action");

if ("list".equals(action)) {
    List<User> users = userDAO.findAll();
    request.setAttribute("users", users);
    request.getRequestDispatcher("users.jsp")
        .forward(request, response);

} else if ("detail".equals(action)) {
    int id = Integer.parseInt(request.getParameter("id"));
    User user = userDAO.findById(id);
    request.setAttribute("user", user);
    request.getRequestDispatcher("user_detail.jsp")
        .forward(request, response);
}

} catch (Exception e) {
    request.setAttribute("error", e.getMessage());
    request.getRequestDispatcher("error.jsp")
        .forward(request, response);
}

}

protected void doPost(HttpServletRequest request,
                      HttpServletResponse response)
    throws ServletException, IOException {

    try {
        String action = request.getParameter("action");

        if ("create".equals(action)) {
            User user = new User();
            user.setName(request.getParameter("name"));
            user.setEmail(request.getParameter("email"));
            user.setPhone(request.getParameter("phone"));

            int newId = userDAO.insert(user);
            response.sendRedirect("/app/user?action=detail&id=" + newId);

        } else if ("update".equals(action)) {
            User user = new User();
            user.setId(Integer.parseInt(request.getParameter("id")));
            user.setName(request.getParameter("name"));
            user.setEmail(request.getParameter("email"));
            user.setPhone(request.getParameter("phone"));

            boolean success = userDAO.update(user);
            if (success) {
                response.sendRedirect("/app/user?action=list");
            }
        }

    } catch (Exception e) {
        request.setAttribute("error", e.getMessage());
    }
}
```

```
        request.getRequestDispatcher("error.jsp")
            .forward(request, response);
    }
}
```

23.4 Factory Pattern cho DAO

```
package com.example.dao;

import com.example.dao.impl.UserDAOImpl;
import com.example.dao.impl.ProductDAOImpl;

public class DAOFactory {

    private static UserDAO userDAO;
    private static ProductDAO productDAO;

    public static UserDAO getUserDAO() {
        if (userDAO == null) {
            userDAO = new UserDAOImpl();
        }
        return userDAO;
    }

    public static ProductDAO getProductDAO() {
        if (productDAO == null) {
            productDAO = new ProductDAOImpl();
        }
        return productDAO;
    }
}
```

Sử dụng Factory

```
UserDAO userDAO = DAOFactory.getUserDAO();
List<User> users = userDAO.findAll();
```

Tóm tắt Session 23

DAO Pattern:

- ☒ Tách database logic khỏi business logic
- ☒ Interface định nghĩa các method
- ☒ Implementation thực hiện JDBC code

- ☒ Dễ bảo trì, test, expand



SESSION 24: MySQL/PostgreSQL Setup

24.1 MySQL Installation

Download MySQL

1. Truy cập <https://www.mysql.com/products/community/>
2. Tải MySQL Server (8.0 hoặc mới hơn)
3. Cài đặt với các option:
 - Server port: 3306 (default)
 - Root password: your_password

Verify Installation

```
# Test connection
mysql -u root -p
# Nhập password

# Hoặc dùng MySQL Workbench
```

24.2 Tạo Database & Table

```
-- 1. Tạo database
CREATE DATABASE mydb;

-- 2. Sử dụng database
USE mydb;

-- 3. Tạo bảng Users
CREATE TABLE users (
  id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
  email VARCHAR(100) UNIQUE NOT NULL,
  phone VARCHAR(20),
  password_hash VARCHAR(255),
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
);

-- 4. Tạo bảng Products
CREATE TABLE products (
  id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(100) NOT NULL,
```

```
description TEXT,
price DECIMAL(10, 2) NOT NULL,
stock INT DEFAULT 0,
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- 5. Tạo bảng Orders
CREATE TABLE orders (
  id INT AUTO_INCREMENT PRIMARY KEY,
  user_id INT NOT NULL,
  order_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  total_price DECIMAL(10, 2),
  status VARCHAR(50) DEFAULT 'pending',
  FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
);

-- 6. Insert dữ liệu mẫu
INSERT INTO users (name, email, phone) VALUES
('Nguyễn Văn A', 'a@example.com', '0912345678'),
('Trần Thị B', 'b@example.com', '0923456789');

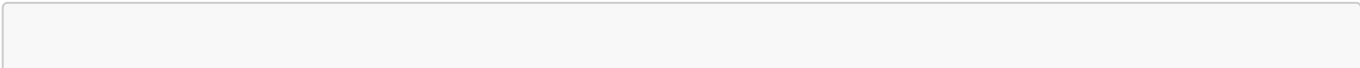
INSERT INTO products (name, price, stock) VALUES
('Laptop Dell', 15000000, 10),
('iPhone 14', 25000000, 5);

-- 7. Xem dữ liệu
SELECT * FROM users;
SELECT * FROM products;
```

24.3 Các Loại Data Type

Type	Mô tả	Ví dụ
INT	Số nguyên	INT, INT AUTO_INCREMENT
VARCHAR(n)	Chuỗi (tối đa n ký tự)	VARCHAR(50), VARCHAR(100)
TEXT	Chuỗi dài	Mô tả sản phẩm
DECIMAL(n,d)	Số thực	DECIMAL(10,2) (10 chữ số, 2 thập phân)
DATE	Ngày	DATE
TIMESTAMP	Ngày giờ	TIMESTAMP DEFAULT CURRENT_TIMESTAMP
BOOLEAN	Đúng/Sai	BOOLEAN (0/1)
BLOB	Dữ liệu nhị phân	Ảnh, file

24.4 Constraints (Ràng buộc)




```
CREATE TABLE users (  
  -- PRIMARY KEY: Khóa chính  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  
  -- NOT NULL: Bắt buộc có giá trị  
  name VARCHAR(100) NOT NULL,  
  
  -- UNIQUE: Giá trị duy nhất  
  email VARCHAR(100) UNIQUE NOT NULL,  
  
  -- DEFAULT: Giá trị mặc định  
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
  
  -- CHECK: Kiểm tra điều kiện  
  age INT CHECK (age >= 0 AND age <= 120),  
  
  -- FOREIGN KEY: Khóa ngoại (liên kết bảng)  
  department_id INT,  
  FOREIGN KEY (department_id) REFERENCES departments(id)  
);
```

24.5 Các Loại Query Thường Dùng

SELECT - Lấy dữ liệu

```
-- Lấy tất cả  
SELECT * FROM users;  
  
-- Lấy cột cụ thể  
SELECT name, email FROM users;  
  
-- WHERE  
SELECT * FROM users WHERE email = 'a@example.com';  
SELECT * FROM users WHERE age > 18;  
  
-- AND, OR  
SELECT * FROM users WHERE age > 18 AND status = 'active';  
SELECT * FROM users WHERE role = 'admin' OR role = 'moderator';  
  
-- ORDER BY  
SELECT * FROM users ORDER BY created_at DESC;  
SELECT * FROM products ORDER BY price ASC;  
  
-- LIMIT  
SELECT * FROM users LIMIT 10;  
SELECT * FROM users LIMIT 10 OFFSET 5; -- Skip 5, lấy 10  
  
-- LIKE (Pattern matching)  
SELECT * FROM users WHERE name LIKE 'Nguyễn%'; -- Bắt đầu với Nguyễn
```

```
SELECT * FROM users WHERE email LIKE '%@example.com'; -- Kết thúc với @example.com

-- COUNT, SUM, AVG
SELECT COUNT(*) FROM users;
SELECT SUM(price) FROM products;
SELECT AVG(price) FROM products;

-- GROUP BY
SELECT category, COUNT(*) FROM products GROUP BY category;

-- JOIN
SELECT u.name, o.order_date, o.total_price
FROM users u
INNER JOIN orders o ON u.id = o.user_id;
```

INSERT - Thêm dữ liệu

```
INSERT INTO users (name, email, phone)
VALUES ('Bùi Văn C', 'c@example.com', '0934567890');

-- Multiple rows
INSERT INTO users (name, email, phone) VALUES
('User 1', 'user1@example.com', '0911111111'),
('User 2', 'user2@example.com', '0922222222');
```

UPDATE - Cập nhật dữ liệu

```
UPDATE users SET email = 'newemail@example.com' WHERE name = 'Nguyễn Văn A';

-- Multiple columns
UPDATE products SET price = 20000000, stock = 0 WHERE id = 1;

-- Remember WHERE clause!
UPDATE users SET status = 'active'; -- ⚠️ Cập nhật TẤT CẢ rows
```

DELETE - Xóa dữ liệu

```
DELETE FROM users WHERE id = 5;

-- Remember WHERE clause!
DELETE FROM users; -- ⚠️ Xóa TẤT CẢ rows!!!
```

24.6 Backup & Restore (mysqldump)

Backup

```
# Backup database
mysqldump -u root -p mydb > mydb_backup.sql

# Backup tất cả databases
mysqldump -u root -p --all-databases > all_databases.sql
```

Restore

```
# Restore database
mysql -u root -p mydb < mydb_backup.sql

# Restore tất cả databases
mysql -u root -p < all_databases.sql
```

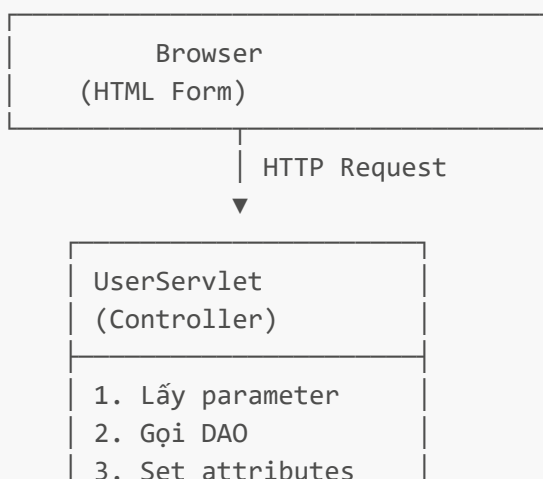
Tóm tắt Session 24

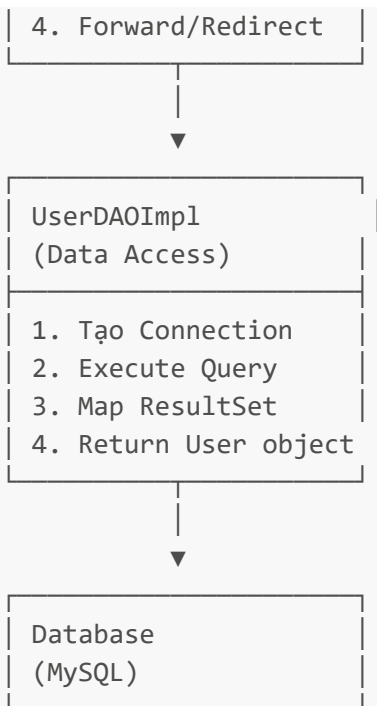
MySQL Setup:

- ☒ Cài MySQL Server
- ☒ Tạo database, table
- ☒ Constraints (PRIMARY KEY, UNIQUE, FOREIGN KEY)
- ☒ SQL queries (SELECT, INSERT, UPDATE, DELETE)
- ☒ Backup/Restore

SESSION 25: Integrate JDBC with Servlet/JSP

25.1 Kiến trúc Complete (MVC + Database)





25.2 Ví dụ: Display User List

users.jsp (View)

```

<%@ page language="java" contentType="text/html; charset=UTF-8" pageEncoding="UTF-8" %>
<%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core" %>

<!DOCTYPE html>
<html>
<head>
  <title>Users</title>
  <style>
    body { font-family: Arial; }
    table { border-collapse: collapse; width: 100%; margin-top: 20px; }
    th, td { border: 1px solid #ddd; padding: 10px; text-align: left; }
    th { background: #f0f0f0; }
  </style>
</head>
<body>
  <h1>Users List</h1>

  <table>
    <thead>
      <tr>
        <th>ID</th>
        <th>Name</th>
        <th>Email</th>
        <th>Phone</th>
        <th>Actions</th>
      </tr>
    </thead>
  </table>

```

```

        </tr>
    </thead>
    <tbody>
        <c:forEach var="user" items="${ users }">
            <tr>
                <td>${ user.id }</td>
                <td>${ user.name }</td>
                <td>${ user.email }</td>
                <td>${ user.phone }</td>
                <td>
                    <a href="/app/user?action=detail&id=${ user.id }">View</a>
                    <a href="/app/user?action=delete&id=${ user.id
} ">Delete</a>
                </td>
            </tr>
        </c:forEach>
    </tbody>
</table>
</body>
</html>

```

UserServlet (Controller)

```

@WebServlet("/user")
public class UserServlet extends HttpServlet {

    private UserDao userDao = DAOFactory.getUserDAO();

    protected void doGet(HttpServletRequest request,
        HttpServletResponse response)
        throws ServletException, IOException {

        String action = request.getParameter("action");

        if ("list".equals(action)) {
            showList(request, response);
        } else if ("detail".equals(action)) {
            showDetail(request, response);
        } else if ("delete".equals(action)) {
            deleteUser(request, response);
        }
    }

    private void showList(HttpServletRequest request,
        HttpServletResponse response)
        throws ServletException, IOException {
        try {
            List<User> users = userDao.findAll();
            request.setAttribute("users", users);
            request.getRequestDispatcher("users.jsp")
                .forward(request, response);
        }
    }
}

```

```
    } catch (Exception e) {
        request.setAttribute("error", "Error: " + e.getMessage());
        request.getRequestDispatcher("error.jsp")
            .forward(request, response);
    }
}

private void showDetail(HttpServletRequest request,
                        HttpServletResponse response)
    throws ServletException, IOException {
    try {
        int id = Integer.parseInt(request.getParameter("id"));
        User user = userDao.findById(id);

        if (user == null) {
            response.sendError(HttpServletResponse.SC_NOT_FOUND);
            return;
        }

        request.setAttribute("user", user);
        request.getRequestDispatcher("user_detail.jsp")
            .forward(request, response);
    } catch (Exception e) {
        request.setAttribute("error", "Error: " + e.getMessage());
        request.getRequestDispatcher("error.jsp")
            .forward(request, response);
    }
}

private void deleteUser(HttpServletRequest request,
                        HttpServletResponse response)
    throws ServletException, IOException {
    try {
        int id = Integer.parseInt(request.getParameter("id"));
        boolean success = userDao.delete(id);

        if (success) {
            response.sendRedirect("/app/user?action=list");
        } else {
            response.sendError(HttpServletResponse.SC_NOT_FOUND);
        }
    } catch (Exception e) {
        request.setAttribute("error", "Error: " + e.getMessage());
        request.getRequestDispatcher("error.jsp")
            .forward(request, response);
    }
}
}
```

25.3 Display với Pagination

```
private void showList(HttpServletRequest request,
                    HttpServletResponse response)
    throws ServletException, IOException {
    try {
        int page = 1;
        int pageSize = 10;

        String pageParam = request.getParameter("page");
        if (pageParam != null && !pageParam.isEmpty()) {
            page = Integer.parseInt(pageParam);
        }

        int offset = (page - 1) * pageSize;

        List<User> users = userDao.findPaginated(offset, pageSize);
        int totalCount = userDao.count();
        int totalPages = (totalCount + pageSize - 1) / pageSize;

        request.setAttribute("users", users);
        request.setAttribute("currentPage", page);
        request.setAttribute("totalPages", totalPages);
        request.setAttribute("totalCount", totalCount);

        request.getRequestDispatcher("users.jsp")
            .forward(request, response);
    } catch (Exception e) {
        e.printStackTrace();
    }
}
```

users.jsp (Có Pagination)

```
<!-- ... table code ... -->

<!-- Pagination -->
<div style="margin-top: 20px;">
    <c:if test="${ currentPage > 1 }">
        <a href="/app/user?action=list&page=${ currentPage - 1 }">< Previous</a>
    </c:if>

    <span>Page ${ currentPage } of ${ totalPages }</span>

    <c:if test="${ currentPage < totalPages }">
        <a href="/app/user?action=list&page=${ currentPage + 1 }">Next →</a>
    </c:if>
</div>
```

Integrate JDBC:

- ☒ Servlet gọi DAO
- ☒ DAO thực thi JDBC query
- ☒ JSP hiển thị kết quả
- ☒ Pagination cho danh sách lớn

SESSION 26: PreparedStatement & SQL Injection

26.1 SQL Injection là gì?

SQL Injection = Kỹ thuật tấn công nhập vào SQL code qua user input.

Ví dụ nguy hiểm

```
// ✗ KHÔNG AN TOÀN
String email = request.getParameter("email");

// Hacker nhập: admin'; DROP TABLE users; --
String sql = "SELECT * FROM users WHERE email = '" + email + "'";
// Câu lệnh thực thi: SELECT * FROM users WHERE email = 'admin'; DROP TABLE users;
--'
```

26.2 PreparedStatement vs Statement

✗ Statement (Không an toàn)

```
String sql = "UPDATE users SET email = '" + email + "' WHERE id = " + id;
Statement stmt = conn.createStatement();
stmt.executeUpdate(sql);

// SQL injection risk!
```

☒ PreparedStatement (An toàn)

```
String sql = "UPDATE users SET email = ? WHERE id = ?";
PreparedStatement pstmt = conn.prepareStatement(sql);
pstmt.setString(1, email); // ? được thay bằng email
pstmt.setInt(2, id);       // ? được thay bằng id
pstmt.executeUpdate();

// Safe! Email và id được treat như dữ liệu, không phải code
```

26.3 PreparedStatement Cách Sử Dụng

Một Parameter

```
String sql = "SELECT * FROM users WHERE email = ?";
PreparedStatement pstmt = conn.prepareStatement(sql);
pstmt.setString(1, "nhan@example.com");

ResultSet rs = pstmt.executeQuery();
```

Nhiều Parameters

```
String sql = "INSERT INTO users (name, email, phone) VALUES (?, ?, ?)";
PreparedStatement pstmt = conn.prepareStatement(sql);
pstmt.setString(1, "Nhân");
pstmt.setString(2, "nhan@example.com");
pstmt.setString(3, "0912345678");

pstmt.executeUpdate();
```

Set Values

```
pstmt.setString(1, value); // String
pstmt.setInt(2, value); // int
pstmt.setDouble(3, value); // double
pstmt.setBoolean(4, value); // boolean
pstmt.setDate(5, value); // java.sql.Date
pstmt.setTimestamp(6, value); // java.sql.Timestamp
pstmt.setNull(7, Types.VARCHAR); // NULL value
```

26.4 Ví dụ: Login với PreparedStatement

```
@WebServlet("/login")
public class LoginServlet extends HttpServlet {

    protected void doPost(HttpServletRequest request,
                          HttpServletResponse response)
        throws ServletException, IOException {

        String email = request.getParameter("email");
        String password = request.getParameter("password");

        String sql = "SELECT id, name, password_hash FROM users WHERE email = ?";
```

```

try (Connection conn = DriverManager.getConnection(url, user, dbPassword);
    PreparedStatement pstmt = conn.prepareStatement(sql)) {

    // Set parameter an toàn
    pstmt.setString(1, email);

    try (ResultSet rs = pstmt.executeQuery()) {
        if (rs.next()) {
            String storedHash = rs.getString("password_hash");

            // Verify password (sử dụng BCrypt)
            if (BCrypt.checkpw(password, storedHash)) {
                // Login success
                HttpSession session = request.getSession();
                session.setAttribute("userId", rs.getInt("id"));
                session.setAttribute("userName", rs.getString("name"));

                response.sendRedirect("/app/home");
            } else {
                // Password sai
                request.setAttribute("error", "Invalid password");
                request.getRequestDispatcher("login.jsp")
                    .forward(request, response);
            }
        } else {
            // Email không tồn tại
            request.setAttribute("error", "Email not found");
            request.getRequestDispatcher("login.jsp")
                .forward(request, response);
        }
    }
} catch (Exception e) {
    request.setAttribute("error", "Database error: " + e.getMessage());
    request.getRequestDispatcher("login.jsp")
        .forward(request, response);
}
}

```

26.5 Kiểm tra Parameter

```

String email = request.getParameter("email");

// 1. Null check
if (email == null || email.isEmpty()) {
    // Error handling
}

// 2. Length check

```

```
if (email.length() > 100) {  
    // Error handling  
}  
  
// 3. Format check (Regex)  
if (!email.matches("[A-Za-z0-9+_.-]+@(.+)$")) {  
    // Error handling  
}  
  
// 4. Database check (Prevent duplicate)  
if (userDAO.findByEmail(email) != null) {  
    // Email already exists  
}
```

26.6 SQL Injection Examples

Safe (PreparedStatement)

```
// Hacker nhập: admin'; --  
String userInput = "admin'; -- ";  
  
String sql = "SELECT * FROM users WHERE username = ?";  
pstmt.setString(1, userInput);  
// Query: SELECT * FROM users WHERE username = 'admin'; -- '  
// Result: No match (treated as literal string)
```

Unsafe (String concatenation)

```
// Hacker nhập: admin'; --  
String userInput = "admin'; -- ";  
  
String sql = "SELECT * FROM users WHERE username = '" + userInput + "'";  
// Query: SELECT * FROM users WHERE username = 'admin'; -- '  
// Result: Truy cập tất cả users (comment hết phần WHERE)
```

Tóm tắt Session 26

SQL Injection Protection:

- ☒ **LUÔN** sử dụng PreparedStatement
- ☒ Không bao giờ nối String cho SQL
- ☒ Kiểm tra parameter input
- ☒ Validate data type

SESSION 27: Connection Pool

27.1 Vấn đề: Tạo Connection Một Cách Lãng Phí

✖ Vấn đề:

Request 1: Tạo connection

├ Request 2: Tạo connection

├ Request 3: Tạo connection

├ Request 4: Tạo connection

└ ... (100 requests)

↓

Tạo 100 connections!!!

Memory không đủ, chậm!!!

☒ Giải pháp: Connection Pool

Connection Pool (10 connections)

├ conn1 (available)

├ conn2 (available)

├ conn3 (used by request 1)

├ conn4 (used by request 2)

├ ...

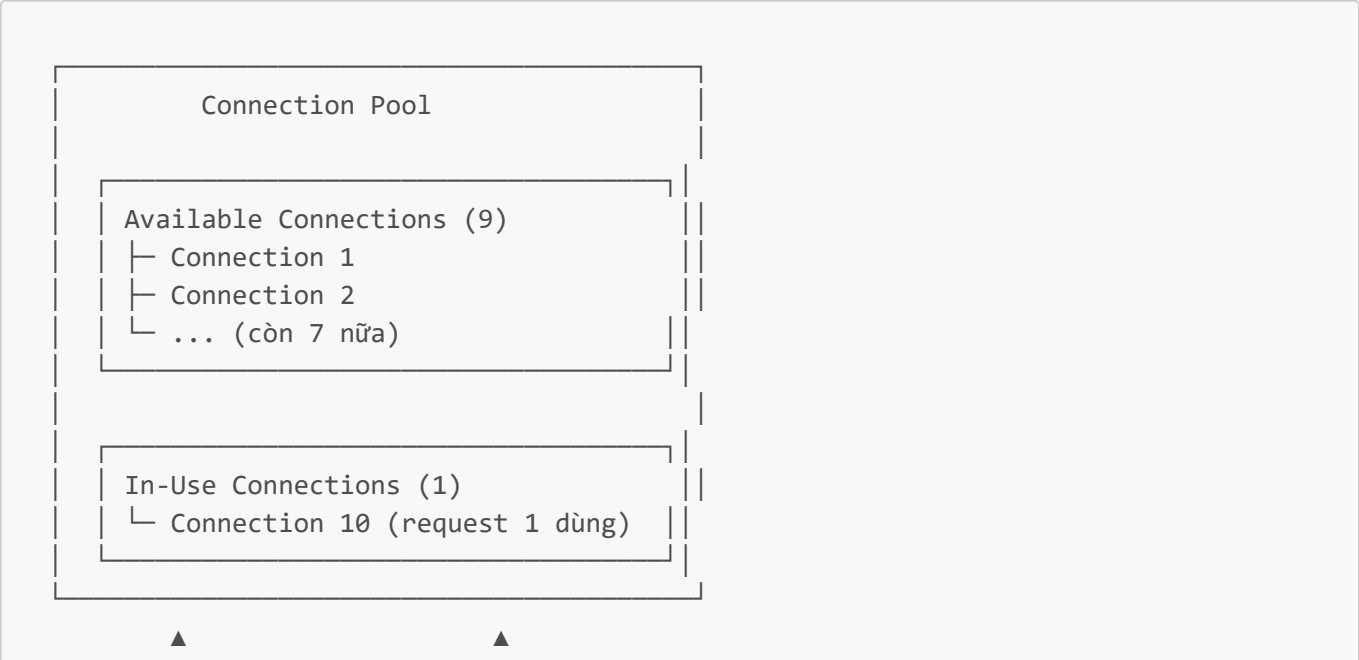
└ conn10 (available)

↓

Tái sử dụng connections

Hiệu suất tốt!

27.2 Connection Pool Cách Hoạt Động



getConnection()	releaseConnection()
(Lấy connection)	(Trả lại connection)

27.3 Apache Commons DBCP

Setup: pom.xml

```
<dependency>
  <groupId>commons-dbcp</groupId>
  <artifactId>commons-dbcp</artifactId>
  <version>1.4</version>
</dependency>
```

Configuration

```
import org.apache.commons.dbcp.BasicDataSource;
import javax.sql.DataSource;

public class DatabaseConnection {

    private static DataSource dataSource;

    static {
        BasicDataSource ds = new BasicDataSource();
        ds.setDriverClassName("com.mysql.cj.jdbc.Driver");
        ds.setUrl("jdbc:mysql://localhost:3306/mydb?
useSSL=false&serverTimezone=UTC");
        ds.setUsername("root");
        ds.setPassword("password");

        // Connection pool settings
        ds.setMinIdle(5);           // Minimum idle connections
        ds.setMaxIdle(10);          // Maximum idle connections
        ds.setMaxActive(20);        // Maximum active connections
        ds.setMaxWait(30000);       // Max wait time (ms)

        dataSource = ds;
    }

    public static Connection getConnection() throws SQLException {
        return dataSource.getConnection();
    }
}
```

Sử dụng

```
// Thay vì:
Connection conn = DriverManager.getConnection(url, user, password);

// Dùng:
Connection conn = DatabaseConnection.getConnection();

// Khi xong:
conn.close(); // Trả lại connection vào pool
```

27.4 Tomcat Connection Pool (Context.xml)

Chỉnh sửa `conf/context.xml` (hoặc `META-INF/context.xml`):

```
<?xml version="1.0" encoding="UTF-8"?>
<Context>
  <!-- Connection Pool Resource -->
  <Resource
    name="jdbc/mydb"
    auth="Container"
    type="javax.sql.DataSource"
    driverClassName="com.mysql.cj.jdbc.Driver"
    url="jdbc:mysql://localhost:3306/mydb?useSSL=false&serverTimezone=UTC"
    username="root"
    password="password"
    maxActive="20"
    maxIdle="10"
    minIdle="5"
    maxWait="30000"
    validationQuery="SELECT 1"
    testOnBorrow="true"
    testOnReturn="true"
    poolPreparedStatements="true"
  />
</Context>
```

web.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app version="4.0" xmlns="...">

  <resource-ref>
    <description>MySQL Database Connection</description>
    <res-ref-name>jdbc/mydb</res-ref-name>
    <res-type>javax.sql.DataSource</res-type>
    <res-auth>Container</res-auth>
  </resource-ref>
```

```
</web-app>
```

Sử dụng Tomcat Pool trong Servlet

```
import javax.naming.Context;
import javax.naming.InitialContext;
import javax.sql.DataSource;

@WebServlet("/user")
public class UserServlet extends HttpServlet {

    private DataSource dataSource;

    @Override
    public void init() throws ServletException {
        try {
            Context ctx = new InitialContext();
            dataSource = (DataSource) ctx.lookup("java:/comp/env/jdbc/mydb");
        } catch (Exception e) {
            throw new ServletException("Cannot find DataSource", e);
        }
    }

    protected void doGet(HttpServletRequest request,
                          HttpServletResponse response)
        throws ServletException, IOException {

        try (Connection conn = dataSource.getConnection();
            Statement stmt = conn.createStatement();
            ResultSet rs = stmt.executeQuery("SELECT * FROM users")) {

            while (rs.next()) {
                System.out.println(rs.getString("name"));
            }
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

27.5 Connection Pool Settings

Setting	Mô tả	Default
minIdle	Số connection tối thiểu	5
maxIdle	Số connection tối đa khi idle	8

Setting	Mô tả	Default
<code>maxActive</code>	Số connection tối đa active	8
<code>maxWait</code>	Thời gian chờ lấy connection (ms)	30000
<code>validationQuery</code>	Query kiểm tra connection	-
<code>testOnBorrow</code>	Kiểm tra connection trước lấy	true
<code>poolPreparedStatements</code>	Cache PreparedStatement	false

27.6 Connection Pool Best Practices

```
// ☒ ĐÚNG
try (Connection conn = dataSource.getConnection();
    PreparedStatement pstmt = conn.prepareStatement(sql)) {

    pstmt.setString(1, value);
    pstmt.executeUpdate();

} catch (SQLException e) {
    // Auto-closed connection
}

// ☒ SAI
Connection conn = dataSource.getConnection();
// ... Quên close → memory leak!
```

Tóm tắt Session 27

Connection Pool:

- ☒ Dừng lại connections (không tạo mới mỗi lần)
- ☒ Apache Commons DBCP hoặc Tomcat pool
- ☒ Cấu hình minIdle, maxActive
- ☒ **LUÔN** try-with-resources để auto-close

SESSION 28: Mini Project - CRUD với Database

28.1 Yêu cầu Dự Án

Tạo **Student Management System** với **Database Backend**:

- Create**: Thêm sinh viên
- Read**: Xem danh sách, chi tiết
- Update**: Cập nhật thông tin

- **Delete:** Xóa sinh viên

Database: MySQL (hoặc PostgreSQL) **Connection:** Connection Pool (DBCP)

28.2 Database Schema

```
-- Tạo database
CREATE DATABASE student_management;
USE student_management;

-- Tạo bảng Students
CREATE TABLE students (
    id INT AUTO_INCREMENT PRIMARY KEY,
    student_id VARCHAR(20) UNIQUE NOT NULL,
    full_name VARCHAR(100) NOT NULL,
    email VARCHAR(100) NOT NULL,
    phone VARCHAR(20),
    major VARCHAR(100),
    year_of_birth INT,
    gpa DECIMAL(3, 2),
    status ENUM('active', 'inactive', 'graduated') DEFAULT 'active',
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
);

-- Insert sample data
INSERT INTO students (student_id, full_name, email, phone, major, year_of_birth,
gpa, status) VALUES
('23001', 'Nguyễn Văn A', 'a@example.com', '0912345678', 'Computer Science', 2005,
3.8, 'active'),
('23002', 'Trần Thị B', 'b@example.com', '0923456789', 'Business', 2005, 3.5,
'active'),
('23003', 'Bùi Văn C', 'c@example.com', '0934567890', 'Engineering', 2006, 3.2,
'active');

-- Verify
SELECT * FROM students;
```

28.3 Project Structure

```
StudentManagementApp/
├─ src/main/java/com/example/
│   │   └─ model/
│   │       └─ Student.java
│   │   └─ dao/
│   │       ├── StudentDAO.java
│   │       └─ StudentDAOImpl.java
│   └─ service/
```

```
|   └─ StudentService.java
└─ servlet/
    └─ StudentServlet.java
        └─ HomeServlet.java
└─ util/
    └─ DatabaseConnection.java
        └─ ValidationUtil.java
└─ controller/
    └─ BaseController.java

└─ src/main/webapp/
    └─ WEB-INF/
        └─ web.xml
            └─ context.xml (nếu dùng Tomcat pool)
    └─ views/
        └─ home.jsp
        └─ student/
            └─ list.jsp
            └─ form.jsp
            └─ detail.jsp
            └─ success.jsp
        └─ error/
            └─ error.jsp
    └─ css/
        └─ style.css
    └─ js/
        └─ validation.js

└─ lib/
    └─ mysql-connector-java-8.0.33.jar
    └─ commons-dbcp-1.4.jar
    └─ jstl-1.2.jar
```

28.4 Model - Student.java

```
package com.example.model;

import java.io.Serializable;

public class Student implements Serializable {
    private static final long serialVersionUID = 1L;

    private int id;
    private String studentId;
    private String fullName;
    private String email;
    private String phone;
    private String major;
    private int yearOfBirth;
    private double gpa;
```

```
private String status;

// Constructors
public Student() {}

public Student(String studentId, String fullName, String email,
               String phone, String major, int yearOfBirth, double gpa) {
    this.studentId = studentId;
    this.fullName = fullName;
    this.email = email;
    this.phone = phone;
    this.major = major;
    this.yearOfBirth = yearOfBirth;
    this.gpa = gpa;
    this.status = "active";
}

// Getters & Setters
public int getId() { return id; }
public void setId(int id) { this.id = id; }

public String getStudentId() { return studentId; }
public void setStudentId(String studentId) { this.studentId = studentId; }

public String getFullName() { return fullName; }
public void setFullName(String fullName) { this.fullName = fullName; }

public String getEmail() { return email; }
public void setEmail(String email) { this.email = email; }

public String getPhone() { return phone; }
public void setPhone(String phone) { this.phone = phone; }

public String getMajor() { return major; }
public void setMajor(String major) { this.major = major; }

public int getYearOfBirth() { return yearOfBirth; }
public void setYearOfBirth(int yearOfBirth) { this.yearOfBirth = yearOfBirth;
}

public double getGpa() { return gpa; }
public void setGpa(double gpa) { this.gpa = gpa; }

public String getStatus() { return status; }
public void setStatus(String status) { this.status = status; }

public int getAge() {
    return 2025 - yearOfBirth;
}
}
```

28.5 DAO - StudentDAO.java

```
package com.example.dao;

import com.example.model.Student;
import java.util.List;

public interface StudentDAO {

    List<Student> findAll() throws Exception;

    Student findById(int id) throws Exception;

    Student findByStudentId(String studentId) throws Exception;

    int insert(Student student) throws Exception;

    boolean update(Student student) throws Exception;

    boolean delete(int id) throws Exception;

    int count() throws Exception;

    List<Student> findPaginated(int offset, int pageSize) throws Exception;
}
```

28.6 DAO Implementation - StudentDAOImpl.java

```
package com.example.dao.impl;

import com.example.dao.StudentDAO;
import com.example.model.Student;
import com.example.util.DatabaseConnection;
import java.sql.*;
import java.util.*;

public class StudentDAOImpl implements StudentDAO {

    @Override
    public List<Student> findAll() throws Exception {
        List<Student> students = new ArrayList<>();
        String sql = "SELECT * FROM students ORDER BY created_at DESC";

        try (Connection conn = DatabaseConnection.getConnection();
            Statement stmt = conn.createStatement();
            ResultSet rs = stmt.executeQuery(sql)) {

            while (rs.next()) {
                students.add(mapResultSetToStudent(rs));
            }
        }
    }
}
```

```
    }
}

return students;
}

@Override
public Student findById(int id) throws Exception {
    String sql = "SELECT * FROM students WHERE id = ?";

    try (Connection conn = DatabaseConnection.getConnection();
        PreparedStatement pstmt = conn.prepareStatement(sql)) {

        pstmt.setInt(1, id);

        try (ResultSet rs = pstmt.executeQuery()) {
            if (rs.next()) {
                return mapResultSetToStudent(rs);
            }
        }

        return null;
    }

}

@Override
public Student findByIdByStudentId(String studentId) throws Exception {
    String sql = "SELECT * FROM students WHERE student_id = ?";

    try (Connection conn = DatabaseConnection.getConnection();
        PreparedStatement pstmt = conn.prepareStatement(sql)) {

        pstmt.setString(1, studentId);

        try (ResultSet rs = pstmt.executeQuery()) {
            if (rs.next()) {
                return mapResultSetToStudent(rs);
            }
        }

        return null;
    }

}

@Override
public int insert(Student student) throws Exception {
    String sql = "INSERT INTO students (student_id, full_name, email, phone,
major, year_of_birth, gpa) " +
        "VALUES (?, ?, ?, ?, ?, ?, ?)";

    try (Connection conn = DatabaseConnection.getConnection();
        PreparedStatement pstmt = conn.prepareStatement(sql,
Statement.RETURN_GENERATED_KEYS)) {
```

```

        pstmt.setString(1, student.getStudentId());
        pstmt.setString(2, student.getFullName());
        pstmt.setString(3, student.getEmail());
        pstmt.setString(4, student.getPhone());
        pstmt.setString(5, student.getMajor());
        pstmt.setInt(6, student.getYearOfBirth());
        pstmt.setDouble(7, student.getGpa());

        pstmt.executeUpdate();

        try (ResultSet generatedKeys = pstmt.getGeneratedKeys()) {
            if (generatedKeys.next()) {
                return generatedKeys.getInt(1);
            }
        }

        return -1;
    }

    @Override
    public boolean update(Student student) throws Exception {
        String sql = "UPDATE students SET full_name = ?, email = ?, phone = ?,
major = ?, " +
            "year_of_birth = ?, gpa = ?, status = ? WHERE id = ?";

        try (Connection conn = DatabaseConnection.getConnection();
            PreparedStatement pstmt = conn.prepareStatement(sql)) {

            pstmt.setString(1, student.getFullName());
            pstmt.setString(2, student.getEmail());
            pstmt.setString(3, student.getPhone());
            pstmt.setString(4, student.getMajor());
            pstmt.setInt(5, student.getYearOfBirth());
            pstmt.setDouble(6, student.getGpa());
            pstmt.setString(7, student.getStatus());
            pstmt.setInt(8, student.getId());

            int rowsAffected = pstmt.executeUpdate();
            return rowsAffected > 0;
        }
    }

    @Override
    public boolean delete(int id) throws Exception {
        String sql = "DELETE FROM students WHERE id = ?";

        try (Connection conn = DatabaseConnection.getConnection();
            PreparedStatement pstmt = conn.prepareStatement(sql)) {

            pstmt.setInt(1, id);

            int rowsAffected = pstmt.executeUpdate();
            return rowsAffected > 0;
        }
    }

```

```
    }
}

@Override
public int count() throws Exception {
    String sql = "SELECT COUNT(*) FROM students";

    try (Connection conn = DatabaseConnection.getConnection();
        Statement stmt = conn.createStatement();
        ResultSet rs = stmt.executeQuery(sql)) {

        if (rs.next()) {
            return rs.getInt(1);
        }
    }

    return 0;
}

@Override
public List<Student> findPaginated(int offset, int pageSize) throws Exception
{
    List<Student> students = new ArrayList<>();
    String sql = "SELECT * FROM students ORDER BY created_at DESC LIMIT ?
OFFSET ?";

    try (Connection conn = DatabaseConnection.getConnection();
        PreparedStatement pstmt = conn.prepareStatement(sql)) {

        pstmt.setInt(1, pageSize);
        pstmt.setInt(2, offset);

        try (ResultSet rs = pstmt.executeQuery()) {
            while (rs.next()) {
                students.add(mapResultSetToStudent(rs));
            }
        }
    }

    return students;
}

private Student mapResultSetToStudent(ResultSet rs) throws SQLException {
    Student student = new Student();
    student.setId(rs.getInt("id"));
    student.setStudentId(rs.getString("student_id"));
    student.setFullName(rs.getString("full_name"));
    student.setEmail(rs.getString("email"));
    student.setPhone(rs.getString("phone"));
    student.setMajor(rs.getString("major"));
    student.setYearOfBirth(rs.getInt("year_of_birth"));
    student.setGpa(rs.getDouble("gpa"));
    student.setStatus(rs.getString("status"));
    return student;
}
```

```
}  
}
```

28.7 Utility - DatabaseConnection.java

```
package com.example.util;  
  
import org.apache.commons.dbcp.BasicDataSource;  
import javax.sql.DataSource;  
import java.sql.Connection;  
import java.sql.SQLException;  
  
public class DatabaseConnection {  
  
    private static DataSource dataSource;  
  
    static {  
        try {  
            BasicDataSource ds = new BasicDataSource();  
            ds.setDriverClassName("com.mysql.cj.jdbc.Driver");  
            ds.setUrl("jdbc:mysql://localhost:3306/student_management?" +  
                    "useSSL=false&serverTimezone=UTC&characterEncoding=UTF-8");  
            ds.setUsername("root");  
            ds.setPassword("password"); // Thay đổi theo mật khẩu của bạn  
  
            // Connection pool settings  
            ds.setMinIdle(5);  
            ds.setMaxIdle(10);  
            ds.setMaxActive(20);  
            ds.setMaxWait(30000);  
  
            dataSource = ds;  
        } catch (Exception e) {  
            e.printStackTrace();  
            throw new RuntimeException("Database initialization failed", e);  
        }  
    }  
  
    public static Connection getConnection() throws SQLException {  
        return dataSource.getConnection();  
    }  
}
```

28.8 Servlet - StudentServlet.java

```
package com.example.servlet;
```



```
import com.example.controller.BaseController;
import com.example.dao.StudentDAO;
import com.example.dao.impl.StudentDAOImpl;
import com.example.model.Student;
import com.example.util.ValidationUtil;
import javax.servlet.ServletException;
import javax.servlet.annotation.WebServlet;
import javax.servlet.http.HttpServletRequest;
import javax.servlet.http.HttpServletResponse;
import java.io.IOException;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

@WebServlet("/student")
public class StudentServlet extends BaseController {

    private StudentDAO studentDAO = new StudentDAOImpl();

    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
        throws ServletException, IOException {

        String action = getStringParam(request, "action");

        try {
            if ("list".equals(action)) {
                getList(request, response);
            } else if ("detail".equals(action)) {
                getDetail(request, response);
            } else if ("form".equals(action)) {
                getForm(request, response);
            } else if ("delete".equals(action)) {
                deleteStudent(request, response);
            } else {
                redirect("/app/student?action=list", response);
            }
        } catch (Exception e) {
            e.printStackTrace();
            request.setAttribute("error", e.getMessage());
            forward("/views/error/error.jsp", request, response);
        }
    }

    @Override
    protected void doPost(HttpServletRequest request, HttpServletResponse
response)
        throws ServletException, IOException {

        String action = getStringParam(request, "action");

        try {
            if ("create".equals(action)) {
                createStudent(request, response);
            }
        }
    }
}
```

```
        } else if ("update".equals(action)) {
            updateStudent(request, response);
        }
    } catch (Exception e) {
        e.printStackTrace();
        request.setAttribute("error", e.getMessage());
        forward("/views/error/error.jsp", request, response);
    }
}

private void getList(HttpServletRequest request, HttpServletResponse response)
    throws Exception {

    int page = getIntParam(request, "page", 1);
    int pageSize = 10;
    int offset = (page - 1) * pageSize;

    List<Student> students = studentDAO.findPaginated(offset, pageSize);
    int totalCount = studentDAO.count();
    int totalPages = (totalCount + pageSize - 1) / pageSize;

    Map<String, Object> attrs = new HashMap<>();
    attrs.put("students", students);
    attrs.put("currentPage", page);
    attrs.put("totalPages", totalPages);
    attrs.put("totalCount", totalCount);

    setAttributes(request, attrs);
    forward("/views/student/list.jsp", request, response);
}

private void getDetail(HttpServletRequest request, HttpServletResponse
response)
    throws Exception {

    int id = getIntParam(request, "id", -1);

    if (id <= 0) {
        response.sendError(HttpServletResponse.SC_BAD_REQUEST);
        return;
    }

    Student student = studentDAO.findById(id);

    if (student == null) {
        response.sendError(HttpServletResponse.SC_NOT_FOUND);
        return;
    }

    request.setAttribute("student", student);
    forward("/views/student/detail.jsp", request, response);
}

private void getForm(HttpServletRequest request, HttpServletResponse response)
```

```
        throws Exception {

String mode = getStringParam(request, "mode");

if ("edit".equals(mode)) {
    int id = getIntParam(request, "id", -1);
    Student student = studentDAO.findById(id);

    if (student == null) {
        response.sendError(HttpServletResponse.SC_NOT_FOUND);
        return;
    }

    request.setAttribute("student", student);
    request.setAttribute("mode", "edit");
} else {
    request.setAttribute("mode", "create");
}

forward("/views/student/form.jsp", request, response);
}

private void createStudent(HttpServletRequest request, HttpServletResponse
response)
    throws Exception {

String fullName = getStringParam(request, "fullName");
String studentId = getStringParam(request, "studentId");
String email = getStringParam(request, "email");
String phone = getStringParam(request, "phone");
String major = getStringParam(request, "major");
String yearStr = getStringParam(request, "yearOfBirth");
String gpaStr = getStringParam(request, "gpa");

// Validation
Map<String, String> errors = new HashMap<>();

if (fullName.isEmpty()) errors.put("fullName", "Full name is required");
if (studentId.isEmpty()) errors.put("studentId", "Student ID is
required");

if (!studentId.isEmpty()) {
    Student existing = studentDAO.findById(studentId);
    if (existing != null) {
        errors.put("studentId", "Student ID already exists");
    }
}

String emailError = ValidationUtil.validateEmail(email);
if (emailError != null) errors.put("email", emailError);

if (!errors.isEmpty()) {
    request.setAttribute("errors", errors);
    request.setAttribute("fullName", fullName);
}
```

```
        request.setAttribute("studentId", studentId);
        request.setAttribute("email", email);
        request.setAttribute("phone", phone);
        request.setAttribute("major", major);
        request.setAttribute("mode", "create");
        forward("/views/student/form.jsp", request, response);
        return;
    }

    // Create student
    try {
        int year = Integer.parseInt(yearStr);
        double gpa = Double.parseDouble(gpaStr);

        Student student = new Student(studentId, fullName, email, phone,
major, year, gpa);
        int newId = studentDAO.insert(student);

        response.sendRedirect("/app/student?action=detail&id=" + newId);
    } catch (NumberFormatException e) {
        request.setAttribute("error", "Invalid year or GPA format");
        forward("/views/student/form.jsp", request, response);
    }
}

private void updateStudent(HttpServletRequest request, HttpServletResponse
response)
    throws Exception {

    int id = getIntParam(request, "id", -1);

    if (id <= 0) {
        response.sendError(HttpServletResponse.SC_BAD_REQUEST);
        return;
    }

    Student student = studentDAO.findById(id);

    if (student == null) {
        response.sendError(HttpServletResponse.SC_NOT_FOUND);
        return;
    }

    // Update from form
    student.setFullName(getStringParam(request, "fullName"));
    student.setEmail(getStringParam(request, "email"));
    student.setPhone(getStringParam(request, "phone"));
    student.setMajor(getStringParam(request, "major"));
    student.setYearOfBirth(getIntParam(request, "yearOfBirth", 2005));
    student.setGpa(Double.parseDouble(getStringParam(request, "gpa")));
    student.setStatus(getStringParam(request, "status"));

    boolean success = studentDAO.update(student);
```

```

        if (success) {
            response.sendRedirect("/app/student?action=detail&id=" + id);
        } else {
            request.setAttribute("error", "Update failed");
            forward("/views/student/form.jsp", request, response);
        }
    }

    private void deleteStudent(HttpServletRequest request, HttpServletResponse
response)
        throws Exception {

        int id = getIntParam(request, "id", -1);

        if (id <= 0) {
            response.sendError(HttpServletResponse.SC_BAD_REQUEST);
            return;
        }

        boolean deleted = studentDAO.delete(id);

        if (deleted) {
            response.sendRedirect("/app/student?action=list&message=deleted");
        } else {
            response.sendError(HttpServletResponse.SC_NOT_FOUND);
        }
    }
}

```

28.9 View - list.jsp

```

<%@ page language="java" contentType="text/html; charset=UTF-8" pageEncoding="UTF-
8" %>
<%@ taglib prefix="c" uri="http://java.sun.com/jsp/jstl/core" %>

<!DOCTYPE html>
<html>
<head>
    <title>Student Management</title>
    <style>
        * { margin: 0; padding: 0; }
        body { font-family: Arial; background: #f5f5f5; padding: 20px; }
        .container { max-width: 1200px; margin: 0 auto; background: white;
padding: 30px; border-radius: 8px; }
        h1 { color: #333; margin-bottom: 20px; }
        .controls { margin-bottom: 20px; }
        .btn { padding: 10px 20px; border: none; border-radius: 4px; cursor:
pointer; background: #007bff; color: white; }
        .btn:hover { background: #0056b3; }
        .btn-danger { background: #dc3545; }
    </style>

```

```

.btn-danger:hover { background: #c82333; }
table { width: 100%; border-collapse: collapse; }
th, td { padding: 12px; border: 1px solid #ddd; text-align: left; }
th { background: #f0f0f0; }
tr:hover { background: #f9f9f9; }
.pagination { margin-top: 20px; text-align: center; }
.pagination a { margin: 0 5px; padding: 8px 12px; background: #ddd; text-decoration: none; }
.pagination a.active { background: #007bff; color: white; }
</style>
</head>
<body>
  <div class="container">
    <h1><img alt="book icon" data-bbox="225 272 245 288"/> Student Management System</h1>

    <div class="controls">
      <a href="/app/student?action=form&mode=create" class="btn">+ New Student</a>
      <a href="/app/" class="btn" style="background: #6c757d;">Home</a>
    </div>

    <table>
      <thead>
        <tr>
          <th>ID</th>
          <th>Student ID</th>
          <th>Full Name</th>
          <th>Email</th>
          <th>Major</th>
          <th>GPA</th>
          <th>Status</th>
          <th>Actions</th>
        </tr>
      </thead>
      <tbody>
        <c:choose>
          <c:when test="${empty students}">
            <tr>
              <td colspan="8" style="text-align: center; color: #999;">No students found</td>
            </tr>
          </c:when>
          <c:otherwise>
            <c:forEach var="s" items="${students}">
              <tr>
                <td>${s.id}</td>
                <td>${s.studentId}</td>
                <td>
                  <a href="/app/student?action=detail&id=${s.id}"
                     style="color: #007bff; text-decoration: none;">
                    ${s.fullName}
                  </a>
                </td>
              </tr>
            </c:forEach>
          </c:otherwise>
        </c:choose>
      </tbody>
    </table>
  </div>
</body>
</html>

```

```

        </td>
        <td>${s.email}</td>
        <td>${s.major}</td>
        <td>${s.gpa}</td>
        <td>
            <span style="padding: 4px 8px; border-radius:
3px;
                                background: ${s.status ==
'active' ? '#28a745' : '#ffc107'};
                                color: white; font-size: 12px;">
                ${s.status}
            </span>
        </td>
        <td>
            <a href="/app/student?
action=form&mode=edit&id=${s.id}"
                                class="btn" style="padding: 5px 10px;
background: #ffc107;">Edit</a>
            <a href="/app/student?
action=delete&id=${s.id}"
                                class="btn btn-danger" style="padding: 5px
10px;"
                                onclick="return confirm('Delete this
student?');">Delete</a>
        </td>
    </tr>
</c:forEach>
</c:otherwise>
</c:choose>
</tbody>
</table>

<div class="pagination">
    <c:if test="${currentPage > 1}">
        <a href="/app/student?action=list&page=${currentPage - 1}">←
Previous</a>
    </c:if>

    <span>Page ${currentPage} of ${totalPages}</span>

    <c:if test="${currentPage < totalPages}">
        <a href="/app/student?action=list&page=${currentPage + 1}">Next
→</a>
    </c:if>
</div>

    <p style="margin-top: 20px; color: #666;">Total: <strong>${totalCount}
</strong> students</p>
</div>
</body>
</html>

```

28.10 Deploy & Test

1. Chuẩn bị MySQL

```
-- Execute SQL script
mysql -u root -p

-- Paste schema từ mục 28.2
```

2. Update DatabaseConnection.java

```
ds.setPassword("your_mysql_password"); // Thay đổi mật khẩu
```

3. Build & Deploy

```
# Sử dụng Maven hoặc tạo WAR bằng IDE
jar cvf StudentManagementApp.war -C src/main/webapp .

# Copy to Tomcat
cp StudentManagementApp.war /path/to/tomcat/webapps/

# Restart Tomcat
```

4. Access Application

```
http://localhost:8080/StudentManagementApp/
```

Hoàn thành Tuần 4!

Bạn đã học:

- ☒ JDBC Basics (Connection, Statement, ResultSet)
- ☒ DAO Pattern (Interface + Implementation)
- ☒ MySQL/PostgreSQL Setup (Database, Schema)
- ☒ Integrate JDBC with Servlet/JSP
- ☒ PreparedStatement & SQL Injection Prevention
- ☒ Connection Pool (Apache DBCP)
- ☒ Mini Project - Full CRUD with Database

Bước tiếp theo: Tuần 5 - Authentication & Security



Tài liệu Tham Khảo

- Oracle JDBC API Documentation
- Apache Commons DBCP Documentation
- MySQL Workbench
- SQL Best Practices
- OWASP SQL Injection Prevention

Tài liệu được biên soạn bởi: Senior Java Developer Ngày cập nhật: 2025 Phiên bản: 1.0